CLASS 6 · THEORY

Search API and Filters in Our Django CRM

How an API finds the right customer or bill using the fields this project already has.

Example queries in this deck describe planned behavior. Search has not been implemented yet.

CRM

TODAY’S LESSON

The questions we will answer

Search API

How does a client ask the Registration API for matching customers?

Text search

When should we use a partial name, a name prefix, or an exact email?

Field filters

How do dates, bill status, amount, and customer ID narrow a result?

Today is about expected behavior. The code lesson comes after we agree on the rules.

OUR ACTUAL PROJECT

What can we search today?

Registration

`name`, `email`, and `date_of_birth` are useful for customer search. The password field is not.

Bill

`bill_id`, `bill_status`, `bill_amount`, `created_at`, and `customer_id` support precise filters.

The JWT-protected `/account/api/registrations/` endpoint lists customers, but has no search configuration. The customer page has a visual search box that does nothing yet. There is no Bill API in the router.

WHY WE NEED IT

The current API makes the client search the whole list

Imagine 50,000 customers and a user trying to find **Rahim Ahmed**. Today the Registration API returns the normal list even if the client sends `?search=rahim`. The client must download or page through many records and search them itself.

User asks for Rahim → **Server returns all** → **Client searches**

> Our target: the server finds matching customers first, then sends a small, useful JSON result.

A Django HTML view can also filter data. We teach the API form because this class already uses DRF and JSON clients.

CORE IDEA

A search API is a list API with a question

A client sends a **GET request** with a search term. The server returns matching records as JSON.

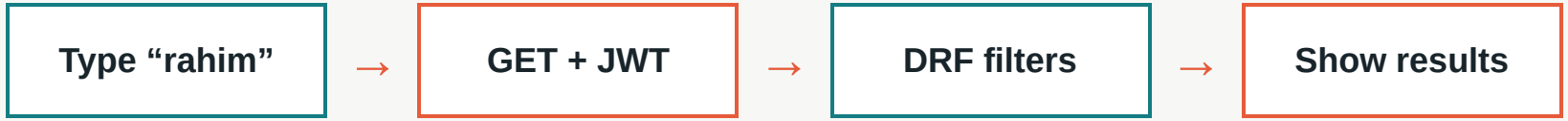
```
GET /account/api/registrations/?search=rahim
```

The path chooses the Registration collection. `search=rahim` asks for customers whose allowed text fields match “rahim”. The same endpoint can still return the full list when no search term is supplied.

> Current behavior: this ViewSet ignores `search`, so the example URL still returns the normal list.

HOW IT IS USED

From the customer search box to matching JSON



The browser can send `GET /account/api/registrations/?search=rahim` with its access token. The ViewSet filters the allowed records, the serializer returns JSON, and the page displays the matching names. The current visual search box must be connected to this flow.

```
Proposed response after safe pagination:
{ "count": 1, "results": [
  { "id": 1, "name": "Rahim Ahmed", "email": "rahim@example.com" }
] }
```

This is a shortened target response. It is not the current API output.

PRODUCTION-GRADE BASICS

A small implementation can still be credible

Correct matches

Search only allowed fields. Exact email and date filters follow clear rules. Invalid input gets an error.

Safe access

Keep JWT permission. Do not search or return passwords or secret bill codes.

Predictable results

Use stable ordering and pagination. Test a match, no match, invalid input, and no token.

Measure slow queries before choosing an index. These basics teach how a product endpoint is built without needing a separate search engine.

READ A URL

The question mark and ampersand

```
/account/api/registrations/?search=rah&born_from=1990-01-01
```

`?` begins the query

Everything before it identifies the API resource. Everything after it gives optional choices.

`&` adds another choice

The request carries both a text search and a birth-date filter. We will define how those conditions combine.

This is a proposed request format. Query parameter names are part of the API design we will choose before coding.

SAMPLE CUSTOMERS

Three records for our class exercises

ID	Name	Email	Date of birth
1	Rahim Ahmed	rahim@example.com	1996-04-12
2	Rahat Islam	rahat@example.com	2000-07-22
3	Ayesha Rahman	ayesha@example.com	1999-01-15

These are fictional values using real `Registration` fields. We will predict results from this table before discussing code.

THREE DIFFERENT JOBS

Search, filter, and ordering

Search

Which customer names contain **rah**?

Filter

Which customers were born before **2000**?

Ordering

Should the matching customers appear by name or ID?

Search and filters decide which records remain. Ordering only arranges those records. Pagination can then show one manageable page.

TEXT SEARCH 1

Contains: find text anywhere in a name

```
Planned request: ?search=rah
```

If we search names with a case-insensitive **contains** rule, all three sample customers match: **Rahim** Ahmed, **Rahat** Islam, and Ayesha **Rah**man.

> The user does not need to remember the complete name or whether “rah” begins the first name or surname.

DRF `SearchFilter` uses case-insensitive contains matching by default when configured for text fields.

TEXT SEARCH 2

Starts with: narrow the name list

```
Search names that start with: rah
```

Rahim Ahmed and **Rahat Islam** match because their names begin with “rah”. **Ayesha Rahman** does not match this rule because her complete name begins with “Ayesha”.

> A prefix rule is useful when the user knows the beginning of the name. It answers a different question from “contains”.

TEXT SEARCH 3

Exact email: identify one customer

```
Planned request: ?email=rahim@example.com
```

The complete email should match **Rahim Ahmed** only. An email address is a specific identifier. If one character is wrong, the API should not guess which customer to edit or bill.

> This is a field filter with an exact rule. It is different from a general `?search=` term that checks several text fields.

DRF SEARCH TOOL

What does SearchFilter provide?

DRF has a built-in **SearchFilter**. After we configure it on the Registration ViewSet and name the allowed fields, it can read a single `search` query parameter and match text in those fields.

It can do

Case-insensitive partial matches by default. It can also use configured starts-with or exact lookups.

It does not do automatically

Typo correction, exact-first relevance ranking, date ranges, or amount ranges.

We will choose `name` and `email` deliberately; sensitive fields do not belong in search.

CUSTOMER FILTERS

Filter by an exact date or a range

Exact date

```
?date_of_birth=1996-04-12
```

From our sample table, Rahim is the only match.

Date range

```
?born_from=1990-01-01
&born_to=1999-12-31
```

Rahim and Ayesha match. Rahat was born in 2000.

Before implementation, we must decide whether range boundaries are included and what response an invalid date should produce.

BILL FILTERS

Use the Bill fields when a Bill API exists

> The current router has no `/account/api/bills/` endpoint. Every request below is a future example.

Request choice	Business question
`?status=no`	Which bills have stored status `no`?
`?min_amount=500&max_amount=2000`	Which bills are within this amount range?
`?customer_id=5`	Which bills belong to Registration ID 5?

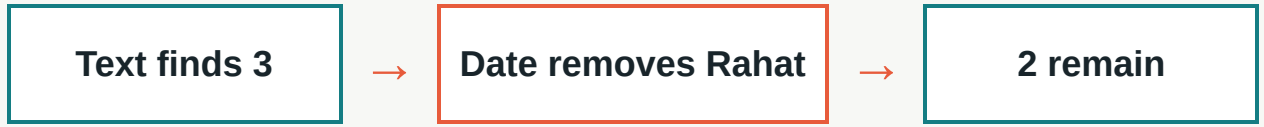
`bill_status` is currently plain text; this project uses `yes` and `no` rather than formal model choices.

COMBINE RULES

Search OR across fields; filter AND by date

```
?search=rah&born_from=1990-01-01&born_to=1999-12-31
```

The general text search may check **name OR email**. The date range then applies as an **AND** condition: a customer must match the text and be born in the range.



The sample result is Rahim Ahmed and Ayesha Rahman.

CLASS EXERCISE

Predict the result before revealing it

Using the three sample customers, answer these:

1. Who matches a name that **starts with** “rah”?
2. Who matches a birth date from **1990 through 1999**?
3. Who satisfies **both** “name contains rah” and that date range?

> Answers: (1) Rahim and Rahat. (2) Rahim and Ayesha. (3) Rahim and Ayesha.

RESULT BEHAVIOR

What if the request has no useful result?

No match

Return an empty list. The interface can say “No customers found.”

Bad value

Explain an invalid date or amount. Do not silently ignore the filter.

Blank search

Choose a consistent rule: show all permitted records or show none.

These decisions are part of the API contract. Students should know the expected response before writing the queryset code.

DATA PROTECTION

Search must follow the API’s access rules

The Registration API already requires a valid JWT access token. Adding a search parameter must not expose records to someone who could not use the list endpoint.

We must allow only intended fields. `Registration.password` and `Bill.bill_code` are not search fields. The current serializer also returns the password field; that separate security issue needs correction before production use.

> A search result should make it easier to find a permitted record, not reveal a sensitive field.

PRACTICAL SCALE

Many results, sorting, and the current cache

If thousands of customers match “a”, returning all of them is difficult to use. A stable sort order and pagination give the client a predictable first page and a way to request more.

The HTML customer page currently caches one **unfiltered** list under `customer_list`. Its search box needs a query-aware design when connected. The DRF Registration API does not automatically use that HTML-page cache.

> Search correctness comes first: each query must return its own matching records.

DRF TOOLBOX

Which tool fits each behavior?

Tool	Role in this project
`SearchFilter`	Built-in DRF text search through `?search=`.
`get_queryset()`	Custom rules for chosen query parameters.
`DjangoFilterBackend`	Structured filters; needs the separate `django-filter` package.
`OrderingFilter`	Sorts results after matching.

`django-filter` is not in this project's `requirements.txt` today. We will choose the simplest tool for the code lesson.

TAKEAWAY

The plan for the code class

1. Make `?search=` work on the existing Registration API.
2. Test partial name search and exact email filtering with known records.
3. Add selected date filters, then handle empty and invalid results.
4. Connect the customer-page input if we also want an HTML demonstration.
5. Later, build a Bill API before demonstrating Bill API filters.

Today we define the behavior. Tomorrow we can trace each request through URL, ViewSet, queryset, serializer, and JSON response.

TEACHER REFERENCES

Official documentation behind this lesson

- DRF: filtering, SearchFilter, OrderingFilter, and DjangoFilterBackend
- DRF: request query parameters

Lesson source: `slides/CLASS_6_SEARCH_DRAFT.md`. Example people and values are fictional; the model fields and current endpoint status come from this project.